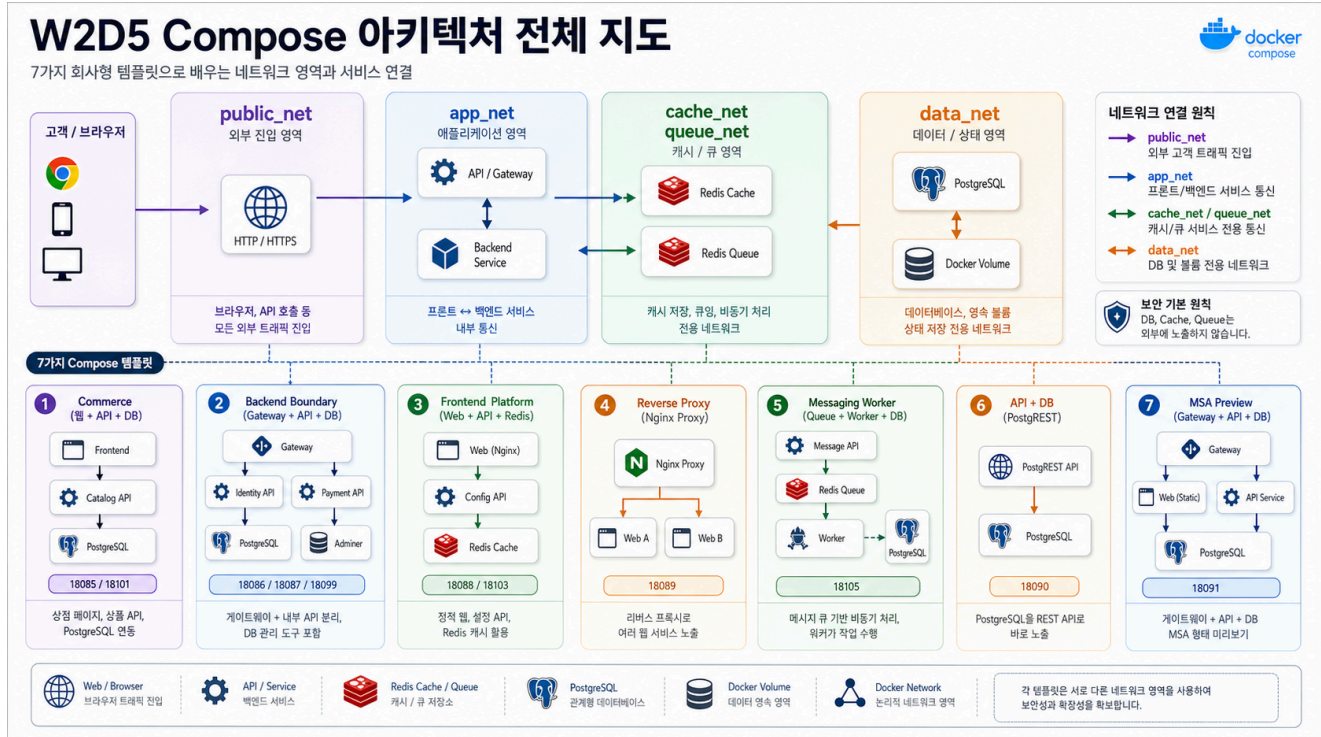


1교시: Compose 기본 개념과 편의성



수업 목표

- 긴 docker run 명령이 왜 인수인계와 재현성을 망가뜨리는지 설명한다.
- compose.yaml의 services, ports, environment, volumes, networks, depends_on을 읽는다.
- 모든 아키텍처 템플릿에 같은 검증 루프를 적용한다.

왜 Compose인가

Day 3~4에서 실행한 명령은 길고, 순서가 중요하고, 사람이 빠뜨리기 쉽다. Web container를 띄우고, DB를 띄우고, network를 맞추고, env를 넣고, volume을 붙이고, log를 확인하는 과정을 매번 손으로 입력하면 팀원이 같은 환경을 재현하기 어렵다.

Compose는 이 실행 조건을 파일로 남긴다. compose.yaml은 단순한 편의 문법이 아니라 local architecture template이다.

docker run에서 하던 일	Compose에서 읽는 위치
image 선택	services.<name>.image 또는 build
host port 공개	ports
runtime config	environment, env_file
data 보존	volumes
service 간 통신	service name, project network
실행 순서 힌트	depends_on

공통 검증 루프

모든 템플릿은 다음 순서로 확인한다.

```
docker compose config
docker compose up -d
docker compose ps
docker compose logs --tail 80
```

docker compose config는 실행 전에 Compose 파일을 정규화해서 보여준다. 강의에서는 이 결과를 먼저 보고 “이 서비스가 어떤 image로 뜨는지, 어떤 port를 host에 공개하는지, 어떤 volume을 붙이는지”를 확인한다.

```
services:
  web:
    image: nginx:1.27-alpine      # 어떤 container image로 실행할지 결정한다.
    ports:
      - "18080:80"              # host 18080 -> container 80으로 공개한다.
    volumes:
      - ./html:/usr/share/nginx/html:ro
                                  # host의 ./html을 container 안 nginx 경로에
읽기 전용으로 붙인다.
    depends_on:
      - api                      # api service를 먼저 시작하라는 실행 순서 힌트
다.
    networks:
      - public_net              # 외부 traffic을 받는 영역
      - app_net                  # backend service와 통신하는 영역

api:
  image: node:20-alpine
```

```

environment:
  DB_HOST: db                                # container끼리는 localhost가 아니라 service
name으로 만난다.
  DB_PORT: "5432"
networks:
  - app_net
  - data_net

db:
  image: postgres:16
  volumes:
    - pgdata:/var/lib/postgresql/data        # DB data는 container가 아니라 named volume에
보존한다.
  networks:
    - data_net                                # DB는 data 영역에만 둔다.

volumes:
  pgdata:                                     # docker compose down만으로는 지워지지 않는다.

networks:
  public_net:
  app_net:
  data_net:

```

이 코드는 암기 대상이 아니라 독해 대상이다. 학생은 ports를 보면 외부 진입점을, environment와 service name을 보면 내부 연결을, volumes를 보면 data lifecycle을 설명할 수 있어야 한다.

네트워크는 “보안 장벽” 하나로 과장하지 않는다. Compose network는 로컬 실습에서 service가 어느 영역에 속하는지 보여주는 구조화 도구다. 운영에서는 firewall, security group, Kubernetes NetworkPolicy, IAM 같은 통제와 함께 생각해야 한다.

서비스별 확인은 architecture마다 다르다.

서비스 유형	확인 예시
Web	<code>curl -I http://localhost:<port></code>
API	<code>curl -s http://localhost:<port>/<resource></code>
DB	<code>docker compose exec db psql ...</code>
Cache	<code>docker compose exec redis redis-cli GET ...</code>
Queue/Worker	<code>docker compose logs worker --tail 40</code>
Proxy	proxy port로 접속하고 upstream service port는 직접 열지 않는다

Cleanup 기준

```
docker compose down
# DB/cache data까지 지워도 되는 실습에서만
# docker compose down -v
```

down은 container와 network를 정리한다. down -v는 named volume까지 지운다. DB가 있는 템플릿에서 down -v는 실습 데이터 삭제다.

핵심 포인트

1교시는 문법 암기 시간이 아니다. 이후 2~8교시에서 자주 쓰는 아키텍처를 바로 실행하기 위해 공통 독해법과 공통 검증 루프를 맞추는 시간이다.

Compose file을 읽을 때는 YAML 줄을 외우지 말고 다음 질문으로 읽는다.

```
외부에서 들어오는 traffic은 어디로 들어오는가?
service끼리는 어떤 이름으로 만나는가?
상태를 가진 service는 어떤 volume을 쓰는가?
실패하면 어떤 logs/exec/curl 결과를 먼저 볼 것인가?
정리할 때 data를 지워도 되는가?
```